INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	Los Hilos (Threads)				
NÚMERO DE LA PRÁCTICA:	01	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	A
FECHA DE PRESENTACIÓN:	15/04/2026	**HORA DE PRESENTACIÓN:**	11:59:00		
INTEGRANTE(s): - Bedregal Pérez Daniel - Jara Mamani Mariel Alisson - Mestas Zegarra Christian Raúl - Noa Camino Yenaro Joel - Sequeiros Condori Luis Gustavo				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS

1. SOLUCIÓN DE EJERCICIOS PROPUESTOS

1.1. Ejercicio 1: Simulación del proceso de cobro en un supermercado

El primer caso práctico aborda la simulación de un supermercado, empezando con una versión de ejecución puramente secuencial. En esta fase, las clases `Cliente` y `Cajera` definen a las entidades. El cliente mantiene un arreglo con los tiempos de espera de sus productos, mientras que la cajera procesa esos tiempos de forma iterativa, simulando el retraso con la función `Thread.sleep()`.

```java
package e1;

public class Cliente {

  private String nombre;
  private int[] carroCompra;

  public Cliente() {}

  public Cliente(String nombre, int[] carroCompra) {
    this.nombre = nombre;
    this.carroCompra = carroCompra;
  }

  public String getNombre() {
    return nombre;
  }

  public void setNombre(String nombre) {
    this.nombre = nombre;
  }

  public int[] getCarroCompra() {
    return carroCompra;
  }

  public void setCarroCompra(int[] carroCompra) {
```

```
    this.carroCompra = carroCompra;
  }
}
```

```
package e1;

public class Cajera {

  private String nombre;

  public Cajera() {}

  public Cajera(String nombre) {
    this.nombre = nombre;
  }

  public String getNombre() {
    return nombre;
  }

  public void setNombre(String nombre) {
    this.nombre = nombre;
  }

  public void procesarCompra(Cliente cliente, long timeStamp) {
    System.out.println(
      "La cajera " +
        this.nombre +
        " COMIENZA A PROCESAR LA COMPRA DEL CLIENTE " +
        cliente.getNombre() +
        " EN EL TIEMPO: " +
        (System.currentTimeMillis() - timeStamp) / 1000 +
        "seg"
    );

    for (int i = 0; i < cliente.getCarroCompra().length; i++) {
      this.esperarXsegundos(cliente.getCarroCompra()[i]);
      System.out.println(
        "Procesado el producto " +
          (i + 1) +
          " ->Tiempo: " +
          (System.currentTimeMillis() - timeStamp) / 1000 +
          "seg"
      );
    }
    System.out.println(
      "La cajera " +
        this.nombre +
        " HA TERMINADO DE PROCESAR " +
        cliente.getNombre() +
        " EN EL TIEMPO: " +
        (System.currentTimeMillis() - timeStamp) / 1000 +
        "seg"
    );
  }

  private void esperarXsegundos(int segundos) {
    try {
      // Pausa la ejecución del hilo actual durante los segundos indicados
      // para simular el costo temporal de escanear un producto.
      Thread.sleep(segundos * 1000);
    } catch (InterruptedException ex) {
      // Reestablece el estado de interrupción si ocurre una excepción durante el reposo
      Thread.currentThread().interrupt();
    }
  }
}
```

Luego, el programa principal `Main` ejecuta este proceso de manera lineal: se procesa al cliente 1 por completo, y cuando la primera cajera termina, se da paso al cliente 2. Esto implica que el tiempo total de ejecución es la suma del tiempo de cobro de todos los clientes.

```java
package e1;

public class Main {

  public static void main(String[] args) {
    Cliente cliente1 = new Cliente("Cliente 1", new int[] { 2, 2, 1, 5, 2, 3 });
    Cliente cliente2 = new Cliente("Cliente 2", new int[] { 1, 3, 5, 1, 1 });
    Cajera cajera1 = new Cajera("Cajera 1");
    Cajera cajera2 = new Cajera("Cajera 2");

    long initialTime = System.currentTimeMillis();
    cajera1.procesarCompra(cliente1, initialTime);
    cajera2.procesarCompra(cliente2, initialTime);
  }
}
```

A continuación, se muestra el resultado de compilar y ejecutar el programa secuencial midiendo su tiempo total con la utilidad `time` de la terminal:

```
❯ javac -d out src/e1/*.java
❯ time java -cp out e1.Main
La cajera Cajera 1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 1 EN EL TIEMPO: 0seg
Procesado el producto 1 ->Tiempo: 2seg
Procesado el producto 2 ->Tiempo: 4seg
Procesado el producto 3 ->Tiempo: 5seg
Procesado el producto 4 ->Tiempo: 10seg
Procesado el producto 5 ->Tiempo: 12seg
Procesado el producto 6 ->Tiempo: 15seg
La cajera Cajera 1 HA TERMINADO DE PROCESAR Cliente 1 EN EL TIEMPO: 15seg
La cajera Cajera 2 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 2 EN EL TIEMPO: 15seg
Procesado el producto 1 ->Tiempo: 16seg
Procesado el producto 2 ->Tiempo: 19seg
Procesado el producto 3 ->Tiempo: 24seg
Procesado el producto 4 ->Tiempo: 25seg
Procesado el producto 5 ->Tiempo: 26seg
La cajera Cajera 2 HA TERMINADO DE PROCESAR Cliente 2 EN EL TIEMPO: 26seg

real            0m26.165s
user            0m0.195s
sys             0m0.069s
```

El análisis del tiempo real reporta un aproximado de 26 segundos, lo cual se corresponde con la suma lineal de las demoras de todos los productos por procesar de ambos clientes ($15 \setminus \texttt{text}\{s\} + 11 \setminus \texttt{text}\{s\}$).

Para aprovechar la multitarea y realizar cobros paralelos, se introduce la clase `CajeraThread`, la cual hereda de la clase `Thread` de Java. En ella se sobreescribe el método `run()`, permitiendo que el procesamiento de la compra se realice en un hilo independiente, de modo que cada cajera procesa su cliente simultáneamente.

```java
package e1;

// Extender Thread permite crear múltiples instancias de esta clase
// que pueden ejecutarse simultáneamente en paralelo
public class CajeraThread extends Thread {

  private String nombre;
  private Cliente cliente;
  private long initialTime;

  public CajeraThread() {}
```

```java
public CajeraThread(String nombre, Cliente cliente, long initialTime) {
  this.nombre = nombre;
  this.cliente = cliente;
  this.initialTime = initialTime;
}

public String getNombre() {
  return nombre;
}

public void setNombre(String nombre) {
  this.nombre = nombre;
}

public long getInitialTime() {
  return initialTime;
}

public void setInitialTime(long initialTime) {
  this.initialTime = initialTime;
}

public Cliente getCliente() {
  return cliente;
}

public void setCliente(Cliente cliente) {
  this.cliente = cliente;
}

// Sobrescribir el método run es el paso esencial para definir
// qué tarea ejecutará este nuevo hilo. Al llamar al método start()
// sobre la instancia de esta clase, el código dentro de run() iniciará.
@Override
public void run() {
  System.out.println(
    "La cajera " +
      this.nombre +
      " COMIENZA A PROCESAR LA COMPRA DEL CLIENTE " +
      this.cliente.getNombre() +
      " EN EL TIEMPO: " +
      (System.currentTimeMillis() - this.initialTime) / 1000 +
      "seg"
  );

  for (int i = 0; i < this.cliente.getCarroCompra().length; i++) {
    this.esperarXsegundos(cliente.getCarroCompra()[i]);
    System.out.println(
      "Procesado el producto " +
        (i + 1) +
        " del cliente " +
        this.cliente.getNombre() +
        "->Tiempo: " +
        (System.currentTimeMillis() - this.initialTime) / 1000 +
        "seg"
    );
  }
  System.out.println(
    "La cajera " +
      this.nombre +
      " HA TERMINADO DE PROCESAR " +
      this.cliente.getNombre() +
      " EN EL TIEMPO: " +
      (System.currentTimeMillis() - this.initialTime) / 1000 +
      "seg"
  );
}

private void esperarXsegundos(int segundos) {
  try {
    Thread.sleep(segundos * 1000);
```

```
    } catch (InterruptedException ex) {
      Thread.currentThread().interrupt();
    }
  }
}
```

El nuevo punto de entrada `MainThread` inicializa y lanza estos hilos empleando el método `start()`.

```
package e1;

public class MainThread {

  public static void main(String[] args) {
    Cliente cliente1 = new Cliente("Cliente 1", new int[] { 2, 2, 1, 5, 2, 3 });
    Cliente cliente2 = new Cliente("Cliente 2", new int[] { 1, 3, 5, 1, 1 });

    long initialTime = System.currentTimeMillis();

    CajeraThread cajera1 = new CajeraThread("Cajera 1", cliente1, initialTime);
    CajeraThread cajera2 = new CajeraThread("Cajera 2", cliente2, initialTime);

    // Inicia la ejecución de los hilos de manera concurrente invocando
    // internamente al método run() de cada instancia.
    cajera1.start();
    cajera2.start();
  }
}
```

La ejecución de esta versión mediante hilos evidencia que las tareas de las cajeras se intercalan. El reporte del comando `time` demuestra un tiempo de ejecución total en torno a los 15 segundos. Esto significa un ahorro de tiempo de casi 11 segundos (alrededor de 42% de ganancia de rendimiento) respecto a la versión secuencial. Al ejecutarse los procesos concurrentemente, la demora total del programa queda limitada únicamente al tiempo del cliente con la compra más larga.

```
❯ javac -d out src/e1/*.java
❯ time java -cp out e1.MainThread
La cajera Cajera 2 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 2 EN EL TIEMPO: 0seg
La cajera Cajera 1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 1 EN EL TIEMPO: 0seg
Procesado el producto 1 del cliente Cliente 2->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2->Tiempo: 11seg
La cajera Cajera 2 HA TERMINADO DE PROCESAR Cliente 2 EN EL TIEMPO: 11seg
Procesado el producto 5 del cliente Cliente 1->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1->Tiempo: 15seg
La cajera Cajera 1 HA TERMINADO DE PROCESAR Cliente 1 EN EL TIEMPO: 15seg

real            0m15.155s
user            0m0.260s
sys             0m0.075s
```

Otra alternativa para lograr el comportamiento concurrente sin utilizar herencia múltiple en Java es la interfaz `Runnable`. En la clase `MainRunnable`, se implementa el método `run()` invocando directamente a la clase `Cajera` original, y se envuelve en nuevos hilos (`Thread`) pasando este `Runnable`.

```
package e1;
```

```java
// Implementar Runnable es una alternativa a heredar de Thread,
// permitiendo compartir el proceso principal en múltiples subprocesos
// y manteniendo libre la capacidad de heredar de otra clase base si hiciera falta.
public class MainRunnable implements Runnable {

  private Cliente cliente;
  private Cajera cajera;
  private long initialTime;

  public MainRunnable(Cliente cliente, Cajera cajera, long initialTime) {
    this.cajera = cajera;
    this.cliente = cliente;
    this.initialTime = initialTime;
  }

  public static void main(String[] args) {
    Cliente cliente1 = new Cliente("Cliente 1", new int[] { 2, 2, 1, 5, 2, 3 });
    Cliente cliente2 = new Cliente("Cliente 2", new int[] { 1, 3, 5, 1, 1 });
    Cajera cajera1 = new Cajera("Cajera 1");
    Cajera cajera2 = new Cajera("Cajera 2");

    long initialTime = System.currentTimeMillis();

    Runnable proceso1 = new MainRunnable(cliente1, cajera1, initialTime);
    Runnable proceso2 = new MainRunnable(cliente2, cajera2, initialTime);

    // Al usar Runnable, es necesario crear una instancia de la clase Thread
    // pasándole como argumento al constructor el objeto Runnable para iniciar su ejecución.
    new Thread(proceso1).start();
    new Thread(proceso2).start();
  }

  // La funcionalidad que el hilo desarrollará, exigido por la interfaz Runnable.
  @Override
  public void run() {
    this.cajera.procesarCompra(this.cliente, this.initialTime);
  }
}
```

El efecto resultante en el sistema es exactamente el mismo en orden y tiempo ($15 \text{\textbackslash text}\{s\}$) que la versión que extiende de `Thread`, demostrando que separar la tarea en una interfaz `Runnable` es una forma eficaz y modular de organizar flujos paralelos sin comprometer el diseño de clases existente.

```
❯ javac -d out src/e1/*.java
❯ time java -cp out e1.MainRunnable
La cajera Cajera 2 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 2 EN EL TIEMPO: 0seg
La cajera Cajera 1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 1 EN EL TIEMPO: 0seg
Procesado el producto 1 ->Tiempo: 1seg
Procesado el producto 1 ->Tiempo: 2seg
Procesado el producto 2 ->Tiempo: 4seg
Procesado el producto 2 ->Tiempo: 4seg
Procesado el producto 3 ->Tiempo: 5seg
Procesado el producto 3 ->Tiempo: 9seg
Procesado el producto 4 ->Tiempo: 10seg
Procesado el producto 4 ->Tiempo: 10seg
Procesado el producto 5 ->Tiempo: 11seg
La cajera Cajera 2 HA TERMINADO DE PROCESAR Cliente 2 EN EL TIEMPO: 11seg
Procesado el producto 5 ->Tiempo: 12seg
Procesado el producto 6 ->Tiempo: 15seg
La cajera Cajera 1 HA TERMINADO DE PROCESAR Cliente 1 EN EL TIEMPO: 15seg

real            0m15.167s
user            0m0.255s
sys             0m0.079s
```

1.2. Ejercicio 2: Problema del Productor y Consumidor

El segundo ejercicio analiza un escenario de interacción controlada de hilos mediante el problema clásico del Productor y el Consumidor. Se utiliza un objeto compartido `CubbyHole` que almacena un entero. Esta clase gestiona el acceso concurrente empleando los monitores propios de Java (`synchronized`) y comunicación condicional (`wait()` y `notifyAll()`) para evitar que el Productor sobreescriba datos o que el Consumidor lea el mismo dato dos veces.

```java
package e2;

// Representa el monitor compartido que garantiza el acceso mutuo.
public class CubbyHole {

  private int contents;
  private boolean available = false;

  // Usa "synchronized" para que solo un hilo acceda a obtener el dato a la vez.
  public synchronized int get() {
    // Patrón de espera condicional: el consumidor espera (wait)
    // hasta que el producto esté disponible. El while previene
    // que despierte erróneamente.
    while (available == false) {
      try {
        wait();
      } catch (InterruptedException e) {}
    }
    available = false;
    // Notifica y despierta a los otros hilos que esperan sobre este monitor.
    notifyAll();
    return contents;
  }

  // Uso de exclusión mutua para el método de depositar producto.
  public synchronized void put(int value) {
    // Si el contenido no se ha consumido, el productor debe esperar.
    while (available == true) {
      try {
        wait();
      } catch (InterruptedException e) {}
    }
    contents = value;
    available = true;

    // Notifica que un nuevo dato ha sido añadido y está listo para leer.
    notifyAll();
  }
}
```

Las clases `Productor` y `Consumidor` extienden de `Thread` y acceden iterativamente a este objeto para poner y obtener valores de 0 a 9. En particular, el hilo Productor intercala un pequeño retraso pseudoaleatorio entre operaciones.

```java
package e2;

// Representa a la entidad que produce elementos, extendiendo Thread
// para ejecutarse en su propia pila.
public class Productor extends Thread {

  private CubbyHole cubbyhole;
  private int numero;

  public Productor(CubbyHole c, int numero) {
    cubbyhole = c;
    this.numero = numero;
  }

  public void run() {
    for (int i = 0; i < 10; i++) {
      // Solicita ingresar el número en el espacio compartido
      cubbyhole.put(i);
      System.out.println("Productor #" + this.numero + "pone:" + i);
```

```
      // Introduce un retraso pseudoaleatorio que puede causar
      // cambios en la intercalación de mensajes al ejecutar.
      try {
        sleep((int) (Math.random() * 100));
      } catch (InterruptedException e) {}
    }
  }
}
```

```
package e2;

// Representa a la entidad que lee elementos secuencialmente.
public class Consumidor extends Thread {

  private CubbyHole cubbyhole;
  private int numero;

  public Consumidor(CubbyHole c, int numero) {
    cubbyhole = c;
    this.numero = numero;
  }

  public void run() {
    int value = 0;
    // Iteraciones para tomar datos que ha introducido el productor.
    for (int i = 0; i < 10; i++) {
      value = cubbyhole.get();
      System.out.println("Consumidor #" + this.numero + " obtiene:" + value);
    }
  }
}
```

El programa `Demo` orquesta la ejecución conjunta, iniciando ambos hilos simultáneamente.

```
package e2;

public class Demo {

  public static void main(String[] args) {
    CubbyHole cub = new CubbyHole();

    // Tanto productor como consumidor referencian a la misma
    // instancia de "CubbyHole", operando concurrentemente
    // sobre los mismos datos.
    Consumidor cons = new Consumidor(cub, 1);
    Productor prod = new Productor(cub, 1);

    prod.start();
    cons.start();
  }
}
```

En la salida del programa se aprecia claramente la coordinación exitosa: el Productor deposita un número y el Consumidor lo recoge sin errores de inconsistencia, ni condiciones de carrera, garantizando un flujo constante pese a la concurrencia.

```
❯ javac -d out src/e2/*.java
❯ time java -cp out e2.Demo
Productor #1pone:0
Consumidor #1 obtiene:0
Productor #1pone:1
Consumidor #1 obtiene:1
Productor #1pone:2
Consumidor #1 obtiene:2
Productor #1pone:3
Consumidor #1 obtiene:3
Productor #1pone:4
Consumidor #1 obtiene:4
Consumidor #1 obtiene:5
Productor #1pone:5
Productor #1pone:6
Consumidor #1 obtiene:6
Productor #1pone:7
Consumidor #1 obtiene:7
Consumidor #1 obtiene:8
Productor #1pone:8
Productor #1pone:9
Consumidor #1 obtiene:9

real            0m0.684s
user            0m0.138s
sys             0m0.068s
```

CUESTIONARIO

1. ¿Por qué es importante el estudio de hilos y multihilos en un sistema distribuido?
   El estudio de hilos en sistemas distribuidos permite aprovechar mejor CPU y red al ejecutar tareas concurrentes por nodo, lo que reduce latencia de respuesta y mejora el rendimiento global del servicio.
2. Describe cómo están compuestos los hilos y cuál es la diferencia entre hilos y procesos
   Un hilo está compuesto por contador de programa, pila, estado de ejecución y registros dentro de un proceso, mientras un proceso tiene espacio de memoria propio y recursos aislados, por lo que los hilos son más ligeros pero comparten memoria y requieren sincronización.
3. Cuadro comparativo de ventajas y desventajas del uso de hilos

Ventajas	Desventajas
Mejor uso de CPU en tareas concurrentes	Requieren control de carrera y exclusión mutua
Menor costo de creación que procesos	Errores de sincronización son difíciles de rastrear
Permiten respuesta más rápida en servicios	Un bloqueo puede afectar a todo el proceso
Comparten datos con menor sobrecarga	Dependen de buen diseño para evitar interbloqueo

El cuadro resume el balance entre costo operativo bajo y mayor complejidad de coordinación cuando varios hilos comparten recursos.

CONCLUSIONES Y RECOMENDACIONES

2. CONCLUSIONES

1. El uso de hilos permite dividir procesos secuenciales largos en tareas que se ejecutan en paralelo, lo que, como se vio en el caso del supermercado, reduce sustancialmente el tiempo total cuando el procesamiento es independiente, mejorando notablemente el rendimiento general de una aplicación concurrente.

2. La comunicación y sincronización son componentes fundamentales para desarrollar sistemas multihilo estables. El uso de bloques `synchronized` junto con los métodos `wait()` y `notifyAll()` en Java, evidenciado en el patrón de productor-consumidor, es esencial para prevenir interbloqueos (deadlocks) y problemas de condición de carrera sobre recursos compartidos.

3. RECOMENDACIONES

1. Se recomienda preferir la implementación de la interfaz `Runnable` sobre la herencia de la clase `Thread` en Java, ya que permite mayor flexibilidad en el diseño orientado a objetos al posibilitar heredar de otra clase distinta si fuera necesario para la lógica de la aplicación.
2. Se debe planificar cuidadosamente la arquitectura de los programas concurrentes identificando en la etapa de diseño cuáles serán las zonas o secciones críticas que requerirán mecanismos explícitos de control de concurrencia y qué tipo de exclusión mutua minimiza la congestión en el sistema sin comprometer su integridad.

REFERENCIAS Y BIBLIOGRAFÍA

[1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.

[2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.

[3] Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.

[4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: Alfaomega Ra-Ma.

[5] Orfali, R., & Harkey, D. (1998). Client/Server Programming with Java and CORBA. USA: Wiley.

[6] Programación multihilo: https://oscarmaestre.github.io/servicios/textos/tema2.html

[7] Threads en Java: https://lc.fie.umich.mx/~rochoa/Materias/PROGRAMACION/PROGRAMACION_2/HILOS.pdf

[8] https://biblus.us.es/bibing/proyectos/abreproy/11320/fichero/Capitulos%252F13.pdf